DOCUMENTO DE GESTIÓN ÁGIL

Documento SCRUM

KarenFlix — Reseñas y ranking de películas, series y anime

Roles · Sprints · Historias de usuario · Seguimiento · Evidencias

ESTUDIANTES

Evelyn Barrios (Persona 1 · Scrum Master)
Wilder Katu (Persona 2 · Product Owner)

DOCENTES EVALUADORES

Eduin Salas
Andre Lopez

INSTITUCIÓN / PROGRAMA

Campuslands · Desarrollo Web Full-Stack / Node.js & Express

PERIODO

26/10/2026 – 30/10/2026

Contenido

Este documento reúne la gestión de **KarenFlix** bajo la metodología SCRUM: una aplicación full-stack (Node.js + Express + MySQL en el backend y HTML, CSS y JavaScript puro en el frontend) para registrar, calificar y rankear películas, series y anime. El trabajo se realiza en pareja durante 2 sprints.

01 **Roles asignados**
Quién ocupa cada rol Scrum y qué responsabilidades asume.

02 **Definición de sprints**
Duración, objetivos, fechas y ceremonias de cada sprint.

03 **Historias de usuario**
Backlog priorizado y fichas con criterios de aceptación.

04 **Herramienta de seguimiento**
Tablero, columnas de flujo y campos registrados por tarea.

05 **Evidencias**
Lista de verificación, capturas del tablero y cierre.

01 Roles asignados

El equipo es una pareja: cada integrante asume un rol Scrum y ambos son developers con módulos completos (backend + frontend).

Product Owner

Responsable: Wilder Katu

- Define y prioriza el backlog
- Redacta las historias de usuario
- Acepta o rechaza los entregables

Scrum Master

Responsable: Evelyn Barrios

- Facilita las ceremonias del sprint
- Identifica y elimina impedimentos
- Vela por el cumplimiento del proceso

Equipo de desarrollo

Responsable: Evelyn Barrios y Wilder Katu

- Estima y ejecuta las tareas
- Entrega incrementos funcionales
- Registra evidencias del sprint

Asignación de roles

Rol	Persona asignada	Sprints en los que actúa	Observaciones
Product Owner	Wilder Katu	Sprint 1 – 2	Prioriza el backlog, redacta las HU y acepta entregables. También desarrolla reseñas, ranking, listados y el frontend principal.
Scrum Master	Evelyn Barrios	Sprint 1 – 2	Dirige la daily, la review y la retro y registra evidencias. También desarrolla auth, catálogo, seguridad y el panel admin.
Equipo de desarrollo	Evelyn Barrios y Wilder Katu	Sprint 1 – 2	Reparto vertical: cada uno hace sus módulos del backend al frontend. Cada PR lo aprueba la otra persona del equipo.
Revisores / docentes	Eduin Salas y Andre Lopez	Sprint 1 – 2	Agregados como colaboradores en los dos repos privados.

Trabajo en pareja

Al final de cada daily, cada integrante explica sus cambios a la otra durante 15 minutos ("**explícame tu PR**"), para que ambos dominen todo el código de punta a punta.

02 Definición de sprints

Describa la duración, los objetivos y las fechas de cada sprint, así como las ceremonias que se realizan.

Parámetros generales

DURACIÓN DEL SPRINT	NÚMERO DE SPRINTS	CAPACIDAD POR SPRINT	FECHA DE INICIO
2 a 3 días	2	32 – 48 h	26/10/2026

Plan de sprints

Sprint	Objetivo del sprint	Inicio	Fin	Estado
Sprint 1	Base técnica, autenticación con roles, catálogo (categorías y películas), seed realista y base visual del frontend. (37 pts)	26/10	27/10	Realizado
Sprint 2	Reseñas con transacciones, likes/dislikes, ranking, frontend completo, panel admin, documentación, despliegue en Vercel y entrega. (46 pts)	28/10	30/10	En proceso

Ceremonias

Planificación
Día 1 y día 3 · 1 hora
Definir el objetivo del sprint, seleccionar historias y repartir subtareas.

Daily
Diario, máximo 15 min
Avance, próximos pasos y bloqueos; después, 15 min de "explícame tu PR".

Revisión
Cierre de cada sprint (días 2 y 5)
Demo de lo terminado en el frontend y en Swagger; se graba como evidencia.

Retrospectiva
Tras la revisión
Qué funcionó, qué mejorar y una acción concreta para el siguiente sprint.

Definición de Hecho (DoD)

- Código en una rama feature y PR aprobado por la otra persona del equipo.
- Criterios de aceptación cumplidos y probados en Bruno.
- Endpoint documentado en Swagger.
- Errores con el código HTTP correcto y mensaje claro en el frontend.
- Sin errores críticos ni credenciales en el repositorio.
- Tarjeta movida a Hecho con su evidencia en el tablero.

03 Historias de usuario

Redacte cada historia con el formato: *Como [rol], quiero [acción], para [beneficio]*. Los criterios de aceptación deben poder verificarse.

Épicas del producto

ID	Épica / módulo	Historias	Puntos	Sprint	Responsable
EP-01	Arquitectura y configuración — Node.js + Express, MySQL, capas, SOLID, errores y SemVer	HU-01, HU-02	8	Sprint 1	Evelyn Barrios
EP-02	Usuarios y seguridad — registro, login JWT, passport, roles, rate limit y CORS	HU-03 – HU-05	11	Sprint 1	Evelyn Barrios
EP-03	Catálogo — categorías, películas y series, aprobación y pósters	HU-06 – HU-08, HU-17	14	Sprint 1 – 2	Evelyn Barrios
EP-04	Reseñas y reacciones — CRUD de reseñas y likes/dislikes con transacciones	HU-11, HU-12	13	Sprint 2	Wilder Katu
EP-05	Ranking y listados — ranking ponderado, filtros, orden y paginación	HU-13, HU-14	10	Sprint 2	Wilder Katu
EP-06	Frontend y experiencia de usuario — base visual, detalle y panel admin	HU-09, HU-15, HU-16	13	Sprint 1 – 2	Wilder Katu · Evelyn Barrios
EP-07	Datos, pruebas y calidad — seed realista, Bruno y casos borde	HU-10, HU-19	5	Sprint 1 – 2	Evelyn Barrios y Wilder Katu
EP-08	Documentación, despliegue y entrega — Swagger, README, Vercel, PDF SCRUM y video	HU-18, HU-20, HU-21	9	Sprint 2	Evelyn Barrios y Wilder Katu

Backlog priorizado

ID	Historia	Prioridad	Puntos	Sprint	Estado
HU-01	Configuración del proyecto y base de datos	Alta	5	Sprint 1	Realizado
HU-02	Arquitectura por capas y manejo de errores	Alta	3	Sprint 1	Realizado

ID	Historia	Prioridad	Puntos	Sprint	Estado
HU-03	Registro de usuarios	Alta	3	Sprint 1	Realizado
HU-04	Inicio de sesión con JWT	Alta	5	Sprint 1	Realizado
HU-05	Roles y protección de rutas	Alta	3	Sprint 1	Realizado
HU-06	Gestión de categorías	Alta	3	Sprint 1	Realizado
HU-07	Registro y gestión de películas y series	Alta	5	Sprint 1	Realizado
HU-08	Aprobación de películas	Alta	3	Sprint 1	Realizado
HU-09	Base visual del frontend y cliente de la API	Alta	5	Sprint 1	En proceso
HU-10	Datos de prueba realistas	Media	2	Sprint 1	Realizado
HU-11	Gestión de reseñas con transacción	Alta	8	Sprint 2	Realizado
HU-12	Like y dislike en reseñas	Alta	5	Sprint 2	Realizado
HU-13	Ranking ponderado	Alta	5	Sprint 2	Realizado
HU-14	Listado con filtros, orden y paginación	Alta	5	Sprint 2	En proceso
HU-15	Detalle de película con reseñas	Alta	3	Sprint 2	En proceso
HU-16	Panel de administración	Alta	5	Sprint 2	Por hacer
HU-17	Subida de pósters	Media	3	Sprint 2	Por hacer
HU-18	Documentación Swagger y README	Alta	3	Sprint 2	Realizado
HU-19	Pruebas y casos borde	Media	3	Sprint 2	Realizado
HU-20	Entrega final	Alta	3	Sprint 2	Por hacer
HU-21	Despliegue en la nube	Alta	3	Sprint 2	Por hacer

Total: 83 puntos de historia · Sprint 1: 37 · Sprint 2: 46 · Evelyn (P1): 38 · Wilder (P2): 36 · Compartidas: 9.

Ficha de historia de usuario

HU-01 · Configuración del proyecto y base de datos	
Historia	**Como desarrolladora, quiero** configurar Node.js + Express y la conexión a MySQL con variables de entorno, **para** tener una base técnica segura y preparada para transacciones.
Criterios de aceptación	☑ Proyecto inicializado con npm y scripts dev/start. ☑ Conexión con mysql2/promise mediante un pool. ☑ Credenciales solo en .env (con .env.example documentado). ☑ schema.sql con claves foráneas, UNIQUE y CHECK. ☑ Los errores de conexión se registran sin exponer datos sensibles.
Subtareas	☑ 01.1 Crear el repo y el package.json con las dependencias obligatorias. ☑ 01.2 Crear config/env.js con dotenv. ☑ 01.3 Crear config/db.js (pool, Singleton) y probar la conexión. ☑ 01.4 Escribir database/schema.sql (tablas normalizadas 3FN). ☑ 01.5 Configurar app.js y server.js (JSON y CORS). ☑ 01.6 Primer commit y rama develop.
Prioridad	Alta
Puntos de historia	5
Sprint asignado	Sprint 1
Responsable	Evelyn Barrios
Estado	**Realizado**

HU-02 · Arquitectura por capas y manejo de errores	
Historia	**Como desarrolladora, quiero** organizar el código en capas aplicando SOLID y patrones de diseño, **para** mantener el código claro, escalable y fácil de explicar.
Criterios de aceptación	☑ Carpetas routes, middlewares, controllers, services, models, config y utils. ☑ Los controladores no tienen SQL y los modelos no tienen lógica de negocio. ☑ Errores centralizados con AppError y errorHandler. ☑ Códigos HTTP correctos (200, 201, 204, 400, 401, 403, 404, 409, 500). ☑ API versionada en /api/v1 y GET /version con el paquete semver.
Subtareas	☑ 02.1 Crear la estructura de carpetas. ☑ 02.2 Implementar utils/AppError.js. ☑ 02.3 Implementar middlewares/errorHandler.js y la respuesta 404. ☑ 02.4 Crear el endpoint /api/v1/version con semver. ☑ 02.5 Configurar ESLint + Prettier.
Prioridad	Alta
Puntos de historia	3
Sprint asignado	Sprint 1
Responsable	Evelyn Barrios
Estado	**Realizado**

HU-03 · Registro de usuarios	
Historia	**Como visitante, quiero** crear una cuenta con nombre, email y contraseña, **para** poder publicar reseñas.
Criterios de aceptación	☑ Email único (409 si ya existe). ☑ Contraseña cifrada con bcrypt. ☑ Validaciones con express-validator (email válido, contraseña de al menos 8 caracteres). ☑ Rol por defecto: usuario. ☑ La respuesta no incluye la contraseña.
Subtareas	☑ 03.1 Modelo de usuarios (create, findByEmail). ☑ 03.2 Service de registro con bcrypt. ☑ 03.3 Validador de registro. ☑ 03.4 Ruta POST /auth/register y su controlador. ☑ 03.5 Prueba en Bruno.
Prioridad	Alta
Puntos de historia	3
Sprint asignado	Sprint 1
Responsable	Evelyn Barrios
Estado	**Realizado**

HU-04 · Inicio de sesión con JWT	
Historia	**Como usuaria registrada, quiero** iniciar sesión y recibir un token, **para** acceder a las funciones protegidas.
Criterios de aceptación	☑ Login con email y contraseña (401 genérico si fallan, sin decir cuál). ☑ Token firmado con jsonwebtoken y expiración definida en .env. ☑ La estrategia passport-jwt protege las rutas privadas (Bearer y Cookie HttpOnly). ☑ GET /auth/me devuelve el perfil de la sesión. ☑ Rate limit más estricto en /auth.
Subtareas	☑ 04.1 Service de login (bcrypt.compare + jwt.sign). ☑ 04.2 config/passport.js con la estrategia JWT. ☑ 04.3 Middleware auth con passport.authenticate. ☑ 04.4 Ruta GET /auth/me. ☑ 04.5 Limitador de peticiones para /auth.
Prioridad	Alta
Puntos de historia	5
Sprint asignado	Sprint 1
Responsable	Evelyn Barrios
Estado	**Realizado**

HU-05 · Roles y protección de rutas	
Historia	**Como administradora, quiero** que solo las cuentas admin gestionen categorías y aprueben películas, **para** proteger la integridad del catálogo.
Criterios de aceptación	☑ Middleware de roles (403 si no hay permiso). ☑ Rutas de administración protegidas con auth + rol. ☑ Rate limit global en /api. ☑ CORS limitado al dominio del frontend. ☑ Usuario admin inicial incluido en el seed.
Subtareas	☑ 05.1 Crear middlewares/roles.js. ☑ 05.2 Aplicar auth + roles a las rutas de admin. ☑ 05.3 Crear middlewares/rateLimit.js (global y auth). ☑ 05.4 Configurar CORS desde .env. ☑ 05.5 Probar los casos 401 y 403.
Prioridad	Alta
Puntos de historia	3
Sprint asignado	Sprint 1
Responsable	Evelyn Barrios
Estado	**Realizado**

HU-06 · Gestión de categorías	
Historia	**Como administradora, quiero** crear, consultar, editar y eliminar categorías, **para** organizar el catálogo (Anime, Ciencia Ficción, Superhéroes, Fantasía).
Criterios de aceptación	☑ GET público; POST, PUT y DELETE solo admin. ☑ Nombre único (409). ☑ No se elimina una categoría con películas asociadas (409). ☑ Validaciones de nombre y descripción.
Subtareas	☑ 06.1 Modelo de categorías. ☑ 06.2 Service con reglas de unicidad y borrado. ☑ 06.3 Validadores. ☑ 06.4 Rutas y controlador. ☑ 06.5 Documentación en Swagger.
Prioridad	Alta
Puntos de historia	3
Sprint asignado	Sprint 1
Responsable	Evelyn Barrios
Estado	**Realizado**

HU-07 · Registro y gestión de películas y series	
Historia	**Como usuaria, quiero** proponer películas, series o anime con título, descripción, categoría, año, tipo, tráiler y actores, **para** ampliar el catálogo.
Criterios de aceptación	☑ Las propuestas quedan en estado pendiente. ☑ Títulos únicos sin importar mayúsculas ni espacios (409). ☑ Año válido y categoría existente. ☑ Soporte para trailer_url y relación de actores en 3FN. ☑ Solo admin edita y elimina. El listado público muestra solo las aprobadas.
Subtareas	☑ 07.1 Modelo de películas con transacciones. ☑ 07.2 utils/normalizar.js para el título. ☑ 07.3 Service de creación con validación de duplicados. ☑ 07.4 Rutas POST, PUT y DELETE. ☑ 07.5 Documentación en Swagger.
Prioridad / Puntos	Alta / 5 pts (Sprint 1 · Evelyn Barrios) · **Estado:** Realizado

HU-08 · Aprobación de películas	
Historia	**Como administradora, quiero** ver las propuestas pendientes y aprobarlas o rechazarlas, **para** mantener la calidad del catálogo.
Criterios de aceptación	☑ GET /peliculas/admin/pendientes solo para admin. ☑ PATCH /peliculas/admin/:id/estado con aprobada o rechazada. ☑ Se registra quién aprobó (aprobado_por). ☑ 404 si la película no existe.
Subtareas	☑ 08.1 Consulta de pendientes en el modelo. ☑ 08.2 Service de cambio de estado. ☑ 08.3 Validador del estado. ☑ 08.4 Rutas y pruebas en Bruno.
Prioridad / Puntos	Alta / 3 pts (Sprint 1 · Evelyn Barrios) · **Estado:** Realizado

HU-09 · Base visual del frontend y cliente de la API	
Historia	**Como usuaria, quiero** una interfaz clara, moderna y responsive con login y registro, **para** usar la app cómodamente desde cualquier dispositivo.
Criterios de aceptación	☑ Repo de frontend separado (HTML, CSS y JS puro). ☑ Estilos globales con variables CSS y diseño mobile-first. ☐ api.js centraliza fetch, token y errores. ☐ Toasts con los mensajes de validación del backend. ☐ Pantallas de login y registro conectadas y menú según el rol.
Subtareas	☑ 09.1 Crear el repo y la estructura (index.html, css/, js/). ☑ 09.2 Hoja de estilos base y componentes (botones, tarjetas, toasts, loader). ☐ 09.3 js/api.js con el manejo del token. ☐ 09.4 Pantallas de login y registro. ☐ 09.5 Navbar dinámico según el rol.
Prioridad	Alta
Puntos de historia	5
Sprint asignado	Sprint 1
Responsable	Wilder Katu
Estado	En proceso

HU-10 · Datos de prueba realistas	
Historia	**Como equipo, quiero** contar con datos variados y realistas, **para** probar el ranking y mostrar una demo convincente.
Criterios de aceptación	☑ 6 categorías y unos 40 títulos de distintos tipos y años. ☑ 12 usuarios (2 admin) con contraseñas cifradas con bcrypt. ☑ Unas 120 reseñas con fechas repartidas en 12 meses y calificaciones variadas. ☑ Reacciones cruzadas, sin likes a reseñas propias. ☑ El script seeds.sql se ejecuta sin errores en MySQL.
Subtareas	☑ 10.1 Lista de títulos y categorías. ☑ 10.2 Generar los hashes bcrypt de los usuarios. ☑ 10.3 Escribir reseñas variadas en tono y longitud. ☑ 10.4 Insertar reacciones coherentes. ☑ 10.5 Archivo database/seeds.sql listo para cargar en MySQL.
Prioridad	Media
Puntos de historia	2
Sprint asignado	Sprint 1
Responsable	Wilder Katu
Estado	**Realizado**

HU-11 · Gestión de reseñas con transacción	
Historia	**Como usuaria, quiero** crear, editar y eliminar mis reseñas con título, comentario y nota de 1 a 10, **para** compartir mi opinión.
Criterios de aceptación	☑ Una reseña por usuaria y película (409). ☑ Calificación entera entre 1 y 10. ☑ Solo la autora edita o elimina (403). ☑ Crear, editar o eliminar recalcula rating_promedio y total_resenas dentro de una transacción. ☑ Si falla un paso se ejecuta ROLLBACK; si todo sale bien, COMMIT.
Subtareas	☑ 11.1 Modelo de reseñas transaccional. ☑ 11.2 Service con beginTransaction, commit y rollback. ☑ 11.3 Validadores. ☑ 11.4 Rutas POST /peliculas/:id/resenas, PUT y DELETE /resenas/:id. ☑ 11.5 Probar el rollback forzando un error.
Prioridad / Puntos	Alta / 8 pts (Sprint 2 · Wilder Katu) · **Estado:** Realizado

HU-12 · Like y dislike en reseñas	
Historia	**Como usuaria, quiero** dar like o dislike a reseñas de otras personas, **para** destacar las opiniones más útiles.
Criterios de aceptación	☑ No se puede reaccionar a la reseña propia (403). ☑ Un voto por persona, que se puede cambiar o quitar. ☑ Los contadores se actualizan en la misma transacción. ☑ La respuesta devuelve los contadores actualizados.
Subtareas	☑ 12.1 Modelo de reacciones (INSERT … ON DUPLICATE KEY UPDATE). ☑ 12.2 Service transaccional. ☑ 12.3 Ruta POST /resenas/:id/reaccion. ☐ 12.4 Botones de like y dislike en el frontend.
Prioridad / Puntos	Alta / 5 pts (Sprint 2 · Wilder Katu) · **Estado:** Realizado

HU-13 · Ranking ponderado	
Historia	**Como visitante, quiero** ver un ranking basado en calificaciones, likes/dislikes y fecha de las reseñas, **para** descubrir los mejores títulos.
Criterios de aceptación	☑ Cada reseña pesa según su recencia y su reputación. ☑ Promedio bayesiano para títulos con pocas reseñas. ☑ GET /ranking?limit=10. ☑ Fórmula documentada en el README.
Subtareas	☑ 13.1 Consulta SQL con cálculo ponderado y bayesiano. ☑ 13.2 Modelo y controlador de ranking. ☑ 13.3 Service y ruta /ranking. ☐ 13.4 Sección Top 10 en Inicio del frontend.
Prioridad / Puntos	Alta / 5 pts (Sprint 2 · Wilder Katu) · **Estado:** Realizado

HU-14 · Listado con filtros, orden y paginación	
Historia	**Como visitante, quiero** listar títulos, filtrar por categoría y ordenar por ranking, popularidad o recientes, **para** encontrar rápido qué ver.
Criterios de aceptación	☑ Filtro por categoría y búsqueda por título (q). ☑ Orden por ranking, popularidad o recientes. ☑ Paginación con un límite máximo de 50. ☐ Pantalla de catálogo con cuadrícula de pósters y chips de categoría.
Subtareas	☑ 14.1 Validar los query params en backend. ☑ 14.2 Consulta SQL con filtros y orden. ☐ 14.3 Pantalla de listado en catalogo.html. ☐ 14.4 Botón Cargar más en catalogo.js.
Prioridad / Puntos	Alta / 5 pts (Sprint 2 · Wilder Katu) · **Estado:** En proceso

HU-15 · Detalle de película con reseñas	
Historia	**Como visitante, quiero** ver la información completa de un título con sus reseñas y tráiler, **para** decidir si lo veo.
Criterios de aceptación	☑ Backend preparado con GET /peliculas/:id y reseñas. ☐ Muestra póster, año, tipo, categoría, tráiler incrustado y reparto de actores. ☐ Lista de reseñas con sus likes y dislikes. ☐ Formulario para reseñar cuando hay sesión (1 a 10).
Subtareas	☑ 15.1 Endpoints probados en backend con trailer_url y actores. ☐ 15.2 Maquetar pantalla de detalle en detalle.html. ☐ 15.3 Formulario con selector de 1 a 10. ☐ 15.4 Mensajes de éxito y error.
Prioridad / Puntos	Alta / 3 pts (Sprint 2 · Wilder Katu) · **Estado:** En proceso

HU-16 · Panel de administración	
Historia	**Como administradora, quiero** un panel para gestionar categorías, películas y propuestas pendientes, **para** administrar todo desde el navegador.
Criterios de aceptación	☑ Backend y middleware de rol admin listos. ☐ Acceso web protegido solo con rol admin. ☐ CRUD de categorías y películas con formularios validados. ☐ Aprobar o rechazar propuestas pendientes.
Subtareas	☐ 16.1 Layout del panel en admin.html y admin.css. ☐ 16.2 Sección de categorías. ☐ 16.3 Sección de películas y trailers. ☐ 16.4 Sección de pendientes con botones de aprobar/rechazar.
Prioridad / Puntos	Alta / 5 pts (Sprint 2 · Evelyn Barrios) · **Estado:** Por hacer

HU-17 · Subida de pósters	
Historia	**Como administradora, quiero** definir la imagen del póster de cada título con vista previa, **para** que el catálogo se vea profesional.
Criterios de aceptación	☑ Validación de URL de póster con express-validator en backend. ☐ El admin puede poner la URL del póster con vista previa en el panel. ☐ Póster por defecto cuando no hay imagen. ☐ Error claro cuando la URL no es válida.
Subtareas	☑ 17.1 Validar imagen_url en backend. ☐ 17.2 Campo de póster con vista previa en el panel. ☐ 17.3 Imagen por defecto (poster-default.webp).
Prioridad / Puntos	Media / 3 pts (Sprint 2 · Evelyn Barrios) · **Estado:** Por hacer

HU-18 · Documentación Swagger y README	
Historia	**Como desarrolladora, quiero** documentar todos los endpoints y el proyecto, **para** que cualquiera pueda instalar y consumir la API.
Criterios de aceptación	☑ Swagger en /api-docs con todos los endpoints, cuerpos, respuestas y autenticación Bearer. ☑ README con descripción, tecnologías, instalación, variables de entorno, estructura, principios, consideraciones técnicas, créditos y ejemplos de endpoints. ☑ Links al repo del frontend y diagramas SVG vectoriales.
Subtareas	☑ 18.1 Crear docs/openapi.json y configuración Swagger. ☑ 18.2 Documentar todos los esquemas de respuestas y códigos HTTP. ☑ 18.3 README técnico formal del backend. ☑ 18.4 Diagramas de capas y arquitectura en SVG.
Prioridad / Puntos	Alta / 3 pts (Sprint 2 · Wilder Katu) · **Estado:** Realizado

HU-19 · Pruebas y casos borde	
Historia	**Como equipo, quiero** probar los flujos principales y los casos borde, **para** entregar un sistema estable.
Criterios de aceptación	☑ Colección de Bruno con 28 peticiones cubriendo todos los endpoints. ☑ Casos borde probados: duplicados (409), calificación inválida (400), like propio (403), token vencido/ausente (401), rol insuficiente (403). ☑ Transacciones verificadas con COMMIT y ROLLBACK. ☑ Export JSON estándar listo para evaluar.
Subtareas	☑ 19.1 Colección de Bruno en bruno/KarenFlix API. ☑ 19.2 Lista de casos borde y archivos de data JSON. ☑ 19.3 Pruebas cruzadas entre Evelyn y Wilder. ☑ 19.4 Exportación de colección JSON lista.
Prioridad / Puntos	Media / 3 pts (Sprint 2 · Evelyn / Wilder) · **Estado:** Realizado

HU-20 · Entrega final	
Historia	**Como equipo, quiero** preparar las evidencias y la entrega, **para** demostrar el funcionamiento y el proceso de desarrollo.
Criterios de aceptación	☑ Repos privados con los trainers (Eduin Salas y Andre Lopez) como colaboradores. ☑ PDF SCRUM con evidencias dentro del repo backend. ☐ CHANGELOG y tag v1.0.0. ☐ Video de máximo 10 minutos con los dos integrantes en cámara. ☑ Conventional Commits en todo el historial de Git.
Subtareas	☑ 20.1 Tablero Kanban actualizado en GitHub Projects. ☑ 20.2 Completar el PDF SCRUM oficial. ☐ 20.3 CHANGELOG y tag v1.0.0. ☐ 20.4 Guion y grabación del video. ☐ 20.5 Revisión final de la entrega.
Prioridad / Puntos	Alta / 3 pts (Sprint 2 · Evelyn / Wilder) · **Estado:** Por hacer

HU-21 · Despliegue en la nube	
Historia	**Como equipo, quiero** publicar KarenFlix en un dominio de Vercel con la base de datos MySQL en la nube, **para** que el profe y cualquier persona puedan usar la app desde un link.
Criterios de aceptación	☐ Frontend publicado en Vercel con un link público. ☐ Backend Express publicado en Vercel; GET /api/v1/version responde en línea. ☐ MySQL en la nube con SSL, cargada con schema.sql y seeds.sql. ☐ Variables de entorno configuradas en Vercel, nunca en el repo. ☐ CORS permite solo el dominio del frontend; links en los dos README.
Subtareas	☐ 21.1 Crear la base MySQL en la nube (Aiven) y cargar esquema y semillas. ☑ 21.2 Agregar src/index.js (export default app) y SSL en config/db.js. ☐ 21.3 Desplegar el backend en Vercel y cargar las variables de entorno. ☐ 21.4 Desplegar el frontend y configurar API_URL en js/config.js. ☐ 21.5 Probar el flujo completo en línea y poner los links en los README.
Prioridad	Alta
Puntos de historia	3
Sprint asignado	Sprint 2
Responsable	Evelyn Barrios y Wilder Katu
Estado	Por hacer

04 Herramienta de seguimiento

Indique la herramienta utilizada para gestionar el tablero, las columnas del flujo y los campos de cada tarea.

Herramienta seleccionada

Herramienta	GitHub Projects (enlazado a issues, PRs y commits)
Enlace al tablero	https://github.com/users/barriosevelyn488-web/projects/1
Tipo de acceso	Público / Colaboradores: barriosevelyn488-web, WilKtu, esalascampuslands
Fecha de creación	26/10/2026

Flujo del tablero (5 columnas profesionales)

Por hacer	En proceso	Testing / En rev.	Realizado	Desplegado
Historias planificadas del Sprint pendientes de inicio.	Tareas en desarrollo activo en ramas feature.	PR abierto, revisión de código ("Explícame tu PR") y Bruno.	PR aprobado y mergeado a develop (cumple DoD).	En producción en Vercel con MySQL cloud.

Campos registrados en cada tarea

Campo	Descripción	Ejemplo
Tipo de tarea	Categoría o naturaleza de la actividad	Backend, frontend, documentación, pruebas
Descripción	Detalle de lo que debe realizarse	Crear POST /api/v1/peliculas/:id/resenas con transacción
Prioridad	Importancia y urgencia de la tarea	Alta
Estado	Situación dentro del flujo de trabajo	Realizado / En proceso
Sprint	Período de desarrollo al que pertenece	Sprint 1 / Sprint 2
Responsable	Persona que ejecuta la tarea	Evelyn Barrios / Wilder Katu
Estimación	Tiempo o puntos estimados	8 puntos / 5 h

05 Evidencias

Marque cada evidencia cuando esté cargada y registre su ubicación para que pueda consultarse.

Lista de verificación

✓	Evidencia	Descripción	Ubicación / enlace
☑	**Repositorio del proyecto**	Enlace al código con historial de commits limpios	repo backend · repo frontend
☑	**Capturas del tablero**	Tablero en cada etapa con tareas y responsables	GitHub Projects (Online)
☑	**Historias de usuario**	Fichas con criterios de aceptación cumplidos	Sección 03 de este documento y GitHub Issues
☐	**Videos de revisión**	Grabación de demostración al cierre (≤ 10 min)	[URL video en README al cierre]
☑	**Actas de retrospectiva**	Mejoras identificadas y aplicadas	Registrada al pie de este documento
☑	**Avance del sprint**	Resumen de puntos completados (14 HUs Hechas)	83 pts totales / 14 HUs en Hecho

Cierre del proyecto (Retrospectiva Ágil)

Qué funcionó

- Arquitectura desacoplada en capas SOLID.
- Transacciones ACID reales en MySQL con rollback ante fallos.
- Seguridad: cookies HttpOnly, rate limit y JWT.
- Base de datos normalizada (3FN) con actores y trailers.
- Documentación interactiva en Swagger UI.

Qué mejorar

- Acordar contratos de URL de endpoints antes de maquetar.
- Asignar mayor margen de tiempo a componentes visuales interactivos de frontend sin librerías.
- Planificar pruebas de integración continuas más frecuentes.

Próximos pasos

- Conectar el panel web de administración con el backend.
- Completar vistas de catálogo y detalle con reproductor de tráiler.
- Aprovisionar MySQL en Aiven y desplegar ambos repos en Vercel.
- Grabar el video final de sustentación en pareja.